



Systems Programming in C—Processes and Signals

In this second article in the series, let us try to understand the concept of processes, loading a program and handling signals in the context of Linux-like operating systems. These concepts are implicitly used even by those who are not programmers, so understanding them will help you get more out of Linux.

A process is defined as a program in execution. We all use the *ps* or *top* commands to get a list of running processes and more information about them. But how do you create a process using the C language interface in Linux? Use the *fork()* system call, as in the *myfork.c* code below.

```
#include<sys/types.h>
#include<stdio.h>

int global =20;
main() {
    int localvar = 30, ret;
    pid_t pid; /* See the Note below */
    ret = fork ();
    if (ret == 0) /*Child Code */ {
        global++; localvar ++;
        exit (0);
    }
    else /*Parent Code */ {
        printf ("My Child's PID =%d, My Id =%d\n", ret, getpid
());
    }
    printf ("My PID= %d, global=%d, local=%d\n", getpid (),
global, localvar);
    exit (0);
}
```



Note: C data-types are normally not used in order to avoid OS or architecture-specific implementation differences. Most data-types used in such programs are available as *typedefs* in the *sys/types.h* header file.

Now what will be the output of the above code? Two lines will be printed, by the two *printfs*, and the values of *localvar* and *global* will be 20 and 30, and not 21 and 31.

And why is this so? The *fork* call creates a new process, which is a copy of the parent. This child process runs independently in a separate address space. It starts executing, not from the *main* function, but after the *fork* call (*ret = fork()*). The *fork* function is different from other calls; it is called once, and returns twice—once in the parent, and once in the child.

Code in each process can distinguish which of the two it is by checking the return value—*fork* returns 0 to the child and a non-zero value (the child's process identifier) to the parent.

The statements incrementing the two variables are actually executed in the child, but since it runs in a separate address space, the variables are not shared. Thus, the last *printf* statement, executed by the parent process, uses the parent's copy of the variables, which are not incremented. So the output is 20 and 30.

The *fork* call is how processes are created and used by most programs. Let's look at a common example—executing an external program from a C program. See the *call_wc.c* code, which you can get from the archive at www.linuxforu.com. In that code, invoke the program *wc* via the function *execlp*; *wc* reads the file *myfile*. The final output will be the number of characters, words, and lines of *myfile*. This is the case when *execlp* is successful—you won't get the last *write* executed. (If *execlp* fails, then the line is executed.) The *execlp* function loads the executable *wc*, passing command-line arguments (*myfile*, *NULL*) to it. *NULL* here means a null pointer, which is used to mark the end of the argument array. The *main* function of a standard C program receives the number of command-line arguments in *argc*, and an array of pointers to individual arguments, represented as strings, in *argv*, as seen in the typical signature *int main (int argc, char *argv [])*.

Now what does *execlp* do? Briefly, it copies into memory the image of the '*wc*' program, overwriting the current process, and sets up the execution environment for the new program. Since the 'old' code (of the current process) no longer exists after a successful invocation of *execlp*, on successful completion, the function *never* returns. This is why you should not get the last-